✨ MODULE 2.1 · C PROGRAMMING NOTES

Decision Making and Branching in C

`if` `if-else` `nested if` `switch-case`

CONTENTS

1. **Conditions in C** — TRUE, FALSE and the operators that build conditions
2. **The if statement** — run code only when a condition is true
3. **The if-else statement** — choose between two paths
4. **Nested if** — a decision inside another decision (+ else-if ladder)
5. **The switch-case statement** — pick one of many options by matching a value
6. **Common mistakes** — the bugs beginners hit most often
7. **Comparison table** — key differences, uses and when to use each

1 Conditions in C

C normally runs statements one after another, from top to bottom. **Decision-making** (branching) statements let the program **choose** what to run, based on a condition. For example: "If `marks >= 40`, print PASS, otherwise print FAIL."

KEY IDEA

A **condition** is an expression that is either **TRUE** or **FALSE**.

In C, `0` means FALSE and **any non-zero value** means TRUE. A comparison such as `5 > 3` gives `1` when it is true and `0` when it is false.

How C reads a condition

Condition	Value	Treated as
`5 > 3`	1	TRUE
`5 == 3`	0	FALSE
`7`	7	TRUE non-zero
`-1`	-1	TRUE non-zero, even if negative
`0`	0	FALSE

OPERATORS USED IN CONDITIONS

Relational operators: compare two values

Operator	Meaning	Example
`==`	equal to	`a == b`
`!=`	not equal to	`num != 0`
`>`	greater than	`a > b`
`<`	less than	`a < b`
`>=`	greater than or equal to	`age >= 18`
`<=`	less than or equal to	`marks <= 100`

Logical operators: combine conditions

Operator	Meaning	Example
`&&`	AND: true only if **both** are true	`x > 0 && y > 0`
`||`	OR: true if **at least one** is true	`x < 0 || y < 0`
`!`	NOT: flips TRUE ↔ FALSE	`!(x > 0)`

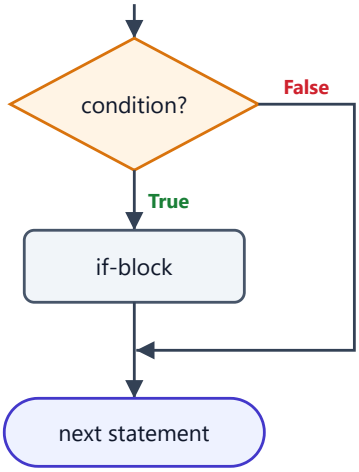
2 The `if` statement

PURPOSE Runs a block of code **only when** the condition is TRUE. If the condition is FALSE, the block is skipped and the program carries on with the next statement.

SYNTAX

```
if (condition) {
    // runs only when condition is TRUE
}
```

FLOWCHART



HOW IT WORKS

TRUE → run the block, then continue with the rest of the program.

FALSE → skip the block, then continue with the rest of the program.

EXAMPLE — Check if a number is positive

```c
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num > 0) {
        printf("%d is positive.\n", num);
    }

    printf("This line always runs.\n");
    return 0;
}
```

OUTPUT

Run 1 · condition TRUE

```
Enter a number: 7
7 is positive.
This line always runs.
```

Run 2 · condition FALSE → if-block skipped

```
Enter a number: -3
This line always runs.
```

REMEMBER

- The condition always goes inside brackets: `if (num > 0)`
- Braces `{ }` are optional when the block has only **one** statement, but always using them avoids mistakes.

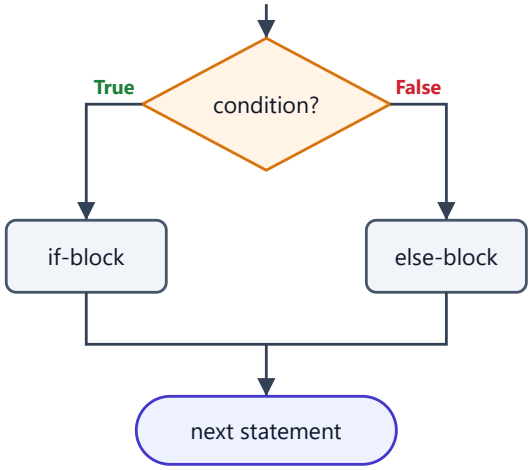
3 The `if-else` statement

PURPOSE Chooses between **two** paths. The if-block runs when the condition is TRUE; the else-block runs when it is FALSE. Exactly **one** of them always runs.

SYNTAX

```
if (condition) {
    // runs when condition is TRUE
} else {
    // runs when condition is FALSE
}
```

FLOWCHART



HOW IT WORKS

TRUE → run the if-block (the else-block is skipped).

FALSE → run the else-block (the if-block is skipped).

EXAMPLE — Check if a number is even or odd

```c
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num % 2 == 0) {
        printf("%d is even.\n", num);
    } else {
        printf("%d is odd.\n", num);
    }

    return 0;
}
```

OUTPUT

Run 1 · condition TRUE

```
Enter a number: 10
10 is even.
```

Run 2 · condition FALSE

```
Enter a number: 7
7 is odd.
```

REMEMBER

- `num % 2` is the remainder when `num` is divided by 2. It is 0 only for even numbers.
- `else` has **no condition** of its own: `else (num < 0)` is wrong. Write `else if (num < 0)` instead (see the else-if ladder in Section 4).
- `else` cannot be used alone. It must come right after an if-block.

4 Nested `if`

PURPOSE An `if` (or `if-else`) written **inside** another `if` or `else`. The inner condition is only checked if the program reaches it, so nesting is used when one decision **depends on** the result of another.

SYNTAX

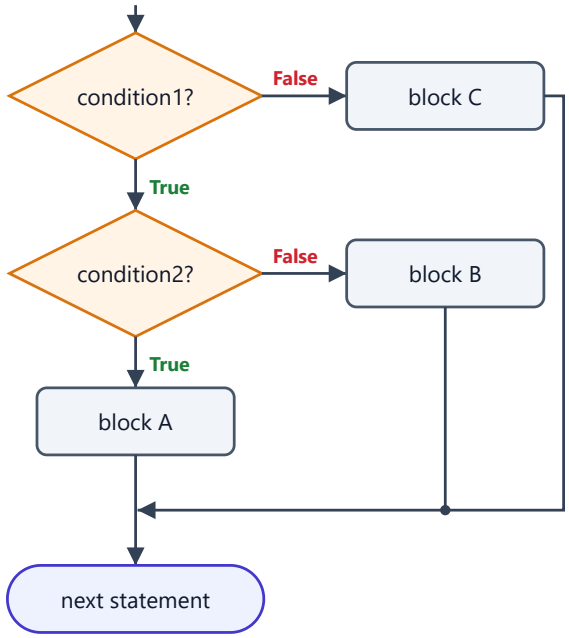
```
if (condition1) {
    if (condition2) {
        // block A
    } else {
        // block B
    }
} else {
    // block C
}
```

A runs when condition1 **and** condition2 are both TRUE.

B runs when condition1 is TRUE but condition2 is FALSE.

C runs when condition1 is FALSE. condition2 is never checked.

FLOWCHART



EXAMPLE — Find the largest of three numbers

```
#include <stdio.h>

int main() {
    int a, b, c;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a > b) {
        if (a > c) {
            printf("%d is the largest.\n", a);
        } else {
            printf("%d is the largest.\n", c);
        }
    } else {
        if (b > c) {
            printf("%d is the largest.\n", b);
        } else {
            printf("%d is the largest.\n", c);
        }
    }

    return 0;
}
```

OUTPUT — with a step-by-step trace

```
Run
Enter three numbers: 12 45 7
45 is the largest.
```

Step	Check	Result	What happens
1	`a > b` → `12 > 45`	FALSE	Go to the outer `else`
2	`b > c` → `45 > 7`	TRUE	Print `b`, which is 45

REMEMBER

- An `else` belongs to the **nearest** `if` above it that does not already have an `else`. Use braces `{ }` so the pairing is always clear.
- Indent each level so you can see which `if` and `else` go together.
- Avoid nesting more than 2–3 levels deep. Joining conditions with `&&` or using an else-if ladder (below) is often simpler.

EXTRA **THE ELSE-IF LADDER** — a flat way to write many choices

When each new `if` sits inside the previous `else`, C lets you write it flat as an **else-if ladder**. Conditions are checked from **top to bottom**: the **first** one that is TRUE runs, and all the rest are skipped. The last `else` runs if none of them are TRUE.

```c
int marks = 68;

if (marks >= 75) {
    printf("Distinction\n");
} else if (marks >= 60) {
    printf("First Class\n");
} else if (marks >= 40) {
    printf("Pass\n");
} else {
    printf("Fail\n");
}
```

```
Output
First Class
```

Check	Result	What happens
`68 >= 75`	FALSE	Go to the next check
`68 >= 60`	TRUE	Print “First Class” and skip the rest

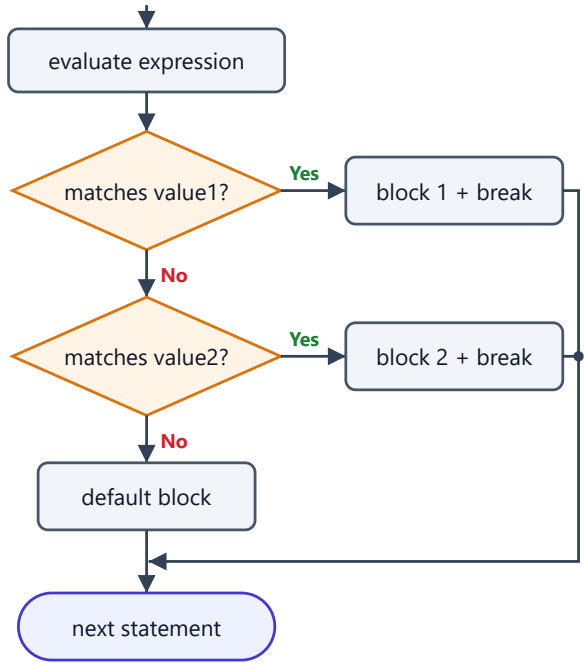
5 The `switch-case` statement

PURPOSE Chooses **one** option out of **many** by matching the value of a single variable (or expression) against fixed values called **cases**. It is a neater replacement for a long else-if ladder that only checks "is x equal to ...?"

SYNTAX

```
switch (expression) {
    case value1:
        // runs when expression == value1
        break;
    case value2:
        // runs when expression == value2
        break;
    default:
        // runs when no case matches
}
```

FLOWCHART



HOW IT WORKS

1. The expression is evaluated **once**.
2. C jumps to the `case` whose value matches.
3. It runs from there until it reaches a `break` (or the end of the switch).
4. If no case matches, `default` runs. If there is no `default`, nothing runs.

WHY BREAK MATTERS: FALL-THROUGH

Without `break`, C does not stop at the end of a case. It **falls through** and runs the cases below it as well.

```
int n = 1;

switch (n) {
    case 1:  printf("One ");
    case 2:  printf("Two ");
    default: printf("Other");
}
```

Output

```
One Two Other
```

We only wanted "**One**". With no `break`, execution fell into `case 2` and `default` too.

FALL-THROUGH ON PURPOSE

Fall-through is useful when several cases should run the **same code**. Stack the cases and write one `break` at the end.

```
case 'a': case 'e': case 'i': case 'o': case 'u':
    printf("Vowel\n");
    break;
```

EXAMPLE — Simple calculator

```c
#include <stdio.h>

int main() {
    int a, b;
    char op;

    printf("Enter an expression (e.g. 5 + 3): ");
    scanf("%d %c %d", &a, &op, &b);

    switch (op) {
        case '+':
            printf("Result = %d\n", a + b);
            break;
        case '-':
            printf("Result = %d\n", a - b);
            break;
        case '*':
            printf("Result = %d\n", a * b);
            break;
        case '/':
            if (b != 0) {
                printf("Result = %d\n", a / b);
            } else {
                printf("Cannot divide by zero!\n");
            }
            break;
        default:
            printf("Invalid operator!\n");
    }

    return 0;
}
```

OUTPUT

Run 1 · matches case '*'

```
Enter an expression (e.g. 5 + 3): 8 * 4
Result = 32
```

Run 2 · case '/', and the if inside it catches b = 0

```
Enter an expression (e.g. 5 + 3): 9 / 0
Cannot divide by zero!
```

Run 3 · no case matches, so default runs

```
Enter an expression (e.g. 5 + 3): 8 x 4
Invalid operator!
```

REMEMBER

- `switch` works only with **integer** values such as `int` and `char`. It does not work with `float`, `double` or strings.
- `case` values must be **constants** (`case 3:` , `case 'A':`). No variables, no ranges like `case marks >= 40:` , and no duplicate values.
- `default` is optional. It is usually written last.

6 Common mistakes

The four bugs beginners hit most often. Each one compiles and runs, but does the wrong thing.

1 Using `=` instead of `==`

✗ WRONG

```
if (x = 5)
```

Stores 5 in `x`, so the condition is always TRUE.

✓ RIGHT

```
if (x == 5)
```

Checks whether `x` is equal to 5.

2 A semicolon right after the condition

✗ WRONG

```
if (x > 0);
{
    printf("Positive");
}
```

The `;` ends the `if` right there, so the block below **always** runs.

✓ RIGHT

```
if (x > 0) {
    printf("Positive");
}
```

No semicolon between `if (...)` and its block.

3 Missing braces around two or more statements

✗ WRONG

```
if (x > 0)
    printf("Positive ");
    printf("number");
```

Only the first `printf` belongs to the `if`. The second one **always** runs, whatever the indentation suggests.

✓ RIGHT

```
if (x > 0) {
    printf("Positive ");
    printf("number");
}
```

The braces make both statements part of the `if`.

4 Forgetting `break` in a switch

✗ WRONG

```
switch (n) {
    case 1: printf("One ");
    case 2: printf("Two ");
}
```

When `n` is 1, this prints `One Two`: execution falls through into `case 2`.

✓ RIGHT

```
switch (n) {
    case 1: printf("One "); break;
    case 2: printf("Two "); break;
}
```

Every case ends with `break`.

TIP

Compile with `gcc -Wall -Wextra file.c` and gcc will warn you about mistakes 1, 2 and 3.

7 Comparison table

All four statements side by side: what each one does, its limits, and when to use it.

Feature	`if`	`if-else`	`nested if`	`switch-case`
Purpose	Runs a block only if the condition is TRUE	Picks one of TWO blocks	Makes a decision inside another decision	Picks one of MANY cases by matching a value
Keywords	`if`	`if` , `else`	`if` , `else` (one inside another)	`switch` , `case` , `break` , `default`
Number of paths	1 (run it or skip it)	2 (exactly one runs)	Many (one per branch)	Many (one per case, plus default)
What it can check	Any condition, using relational and logical operators	Any condition	Any condition (one per level)	Only equality (`==`) with fixed case values
Data types	Any (`int` , `float` , `char` ...)	Any	Any	Only integer types (`int` , `char`)
Ranges like `marks >= 40` ?	Yes	Yes	Yes	No exact values only
If nothing matches	Nothing runs	The `else` block runs	The `else` at that level runs (if written)	`default` runs (if written)
Needs `break` ?	No	No	No	Yes at the end of each case
Readability	Very simple	Simple	Confusing if nested too deep	Neat, even with many options
Example uses	Show a warning only when the battery is low	Even/odd, pass/fail, can vote or not	Largest of three; check username, then password	Menus, calculator, day name from day number
Use it when...	An action is needed in **one** case only	There are exactly **two** outcomes (for 3+ choices, use an else-if ladder)	A check only makes sense **after** an earlier check passes	**One** variable is compared with many **fixed** values